Distributed and Parallel Computing

MPI Report

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The program serial implementation is divided into four main parts:

1.1 ReadInput

A for loop is used to read each line of the instance file and get the number of iterations, users, items, latent features, non zero elements and the value of the convergence coefficient. After this step, the matrix A and a vector of the indexes of the non zero elements are then populated.

1.2 InitialLR

The routine provided was used to initialise the matrices L and R.

1.3 UpdateLR

A recommendation score is predicted for each one of the non zero elements of matrix A, using its i and j indexes to compute the internal product of the row i of the matrix L and the column j of matrix R. Afterwards, the differential between the real value of the non zero element in matrix A and its prediction is computed for all the non zero elements. Finally and also for each one of the non zero elements of matrix A, the selected column of matrix L and a row of R are updated to decrease the obtained differential as a function of the existing differential, the convergence coefficient and the stored matrices L and R. The function is repeated in a loop for the number of iterations and for each iteration the program stores the values of the matrices L and R and uses them to update the matrices L and R.

1.4 FilterFinalMatrix

The matrix B is computed as the multiplication of matrices L and R. Afterwards, for each row of matrix B, the maximum value is computed without taking into account the the non zero elements. The location of the values are stored in a vector and streamed to the standard output.

2 MPI Version

The MPI version uses a *replicate-all* design: every process holds full copies of L and R, and the only data exchanged each iteration is the gradient.

2.1 Foster Methodology

2.1.1 ReadInput

Partitioning. The instance is parsed once, on the root. **Communication.** A validity status is broad-

cast first (so a malformed file aborts every rank together rather than deadlocking), then the header and the non-zero arrays are broadcast to all processes. **Agglomeration and Mapping.** After the broadcast every process holds the same A and non-zero lists.

2.1.2 InitialLR

Partitioning. None: the routine is deterministic (`srandom(1)`). **Communication.** None — because the seed is fixed, every process generates *identical* L and R independently, so the factors never have to be broadcast. **Agglomeration and Mapping.** Each process keeps a full copy of L and R.

2.1.3 UpdateLR

Partitioning. The non-zero list is split into one contiguous block per process (Quinn's `BLOCK_LOW/SIZE`); each process computes the gradient increments for its block only. **Communication.** The partial gradients dL and dR are summed with two `MPI_Allreduce(MPI_IN_PLACE, ..., MPI_SUM)` calls, so each process ends the iteration with the same global increment. **Agglomeration and Mapping.** Each process applies that increment to its own snapshot of the factors, keeping every copy identical.

2.1.4 FilterFinalMatrix

Partitioning. Performed on the root only. **Communication.** None — L and R are already identical everywhere. **Agglomeration and Mapping.** The root forms $B = L \cdot R$, masks the known entries, and prints one recommendation per user.

2.2 Decomposition

The parallelism is a *domain decomposition of the gradient sum*: each non-zero's residual and increment are independent, so the non-zeros are split across processes and their contributions summed. Because the gradient is a sum the partition is arbitrary; we use equal contiguous blocks.

2.3 Synchronization

The two `Allreduce` calls per iteration are the only synchronisation: they are collective, so every process blocks until the global gradient is ready before applying it. The snapshot `StoreL`/`StoreR` guarantees that all reads in an iteration use the pre-iteration factors, so the result is independent of the partition and matches the serial output bit-for-bit at any process count.

3 Optimisations

The replicate-all design is deliberately simple, and two choices keep it both correct and cheap.

3.1 Deterministic init avoids a broadcast

Because `initialLR` is seeded with a fixed value, every process can generate the *same* L and R on its own. The factors — the largest arrays in the program — therefore never travel across the network; only the per-iteration gradient does. A sequential property of the serial code (a fixed RNG seed) becomes a communication saving in the parallel one.

3.2 One `Allreduce` instead of many messages

Summing partial gradients with `MPI_Allreduce` (rather than gathering to the root and scattering back) lets the MPI library use an efficient tree/ring collective and leaves the result already replicated on every process, exactly what the next iteration needs. `MPI_IN_PLACE` avoids a second buffer.

4 Implications for Parallel and Distributed Computing

4.1 Replicate-all trades communication for simplicity

Keeping a full copy of L and R on every process makes the code short and the decomposition trivial, but it has a price: the per-iteration `Allreduce` moves the *whole* gradient, $O(features \cdot (users + items))$ doubles, and that volume does *not* shrink as processes are added. Compute per process falls as $1/P$ while communication per process stays constant, so at some point communication dominates.

4.2 The strong-scaling ceiling

This is the central lesson. On compute-heavy instances the design scales for a while; on *wide* instances (a large R, hence a large gradient) it reaches the ceiling early and can even *anti-scale* — adding processes makes it slower, because the extra ranks add reduction traffic faster than they remove work. Replicate-all is a correct baseline, not a scalable one.

4.3 Memory does not scale either

Every process stores full L, R and A, so the largest solvable problem is bounded by a *single* node's memory, no matter how many nodes are used. A scalable design must distribute the factors, not just the work.

4.4 Towards a scalable design

Both limits point the same way: stop replicating. Arranging the processes as a $P_r \times P_c$ grid and partitioning L by user-rows and R by item-columns makes each process own a slice of size $O((users \cdot features + features \cdot items)/\sqrt{P})$, and replaces the global `Allreduce` with smaller reductions over row and column sub-communicators — so communication *and* memory fall with P. Within each process the gradient can be threaded with OpenMP (a hybrid model), turning the per-thread write conflict into the same array reduction used in the shared-memory version. The replicate-all program is the correctness oracle these refinements are validated against.

4.5 A serial tail still bounds the result

The final $B = L \cdot R$ and arg-max run only on the root, an $O(users \cdot items \cdot features)$ sequential tail. By Amdahl's law it caps the achievable speedup; distributing that stage too (as the grid design does) is what removes the cap.

5 Evaluation

Figure 1: Speedup

Table 1: Speedup

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst0.in	NaN	NaN	NaN	NaN
inst1.in	NaN	NaN	NaN	NaN
inst1000-1000-100-2-30.in	0.9628647	1.753623	3.270270	4.172414
inst2.in	NaN	NaN	NaN	NaN
inst200-10000-50-100-300.in	1.0146163	1.880361	3.470833	6.170370
inst30-40-10-2-10.in	0.7500000	1.500000	1.500000	1.500000
inst400-50000-30-200-500.in	0.7269504	1.567278	2.953890	4.361702
inst500-500-20-2-100.in	0.8542435	1.618881	2.676301	2.858025
inst50000-5000-100-2-5.in	0.8337190	1.446101	2.703108	3.283854
inst600-10000-10-40-400.in	0.8638906	1.497845	2.657744	2.982833
instML100k.in	0.7516242	1.401677	2.588640	2.990060
instML1M.in	0.4093360	1.221217	3.304762	4.180723

In our opinion, our OpenMP implementation improves the speedup when it has more threads working as supposed. Nevertheless, we know by Amdahl's law that the speed is limited by the total time needed for the execution of the sequential part. As our program is not completely parallelized, the speedup cannot increase linearly with the amount of threads.

Figure 2: Total Execution Time

Table 2: Total Execution Time

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst0.in	0	0	0	0
inst1.in	2	2	0	2
inst1000-1000-100-2-30.in	377	207	111	87
inst2.in	0	2	2	2
inst200-10000-50-100-300.in	821	443	240	135
inst30-40-10-2-10.in	8	4	4	4
inst400-50000-30-200-500.in	1410	654	347	235
inst500-500-20-2-100.in	1084	572	346	324
inst50000-5000-100-2-5.in	3025	1744	933	768
inst600-10000-10-40-400.in	1609	928	523	466
instML100k.in	2001	1073	581	503
instML1M.in	5934	1989	735	581